
Arquitecturas y aplicaciones de las redes neuronales convolucionales

PID_00299274

Jordi Casas Roma

Tiempo mínimo de dedicación recomendado: 3 horas



Universitat
Oberta
de Catalunya

**Jordi Casas Roma**

Doctor en Informática (Universitat Autònoma de Barcelona, UAB, 2014), Máster en Inteligencia Artificial Avanzada (Universidad Nacional de Educación a Distancia, UNED, 2011) y Licenciado en Informática (Universitat Autònoma de Barcelona, UAB, 2002). Actualmente, es profesor agregado de la Facultad de Informática, Multimedia y Telecomunicaciones de la Universitat Oberta de Catalunya (UOC). Sus actividades docentes se concentran principalmente en minería de datos, aprendizaje automático, *big data* y minería de gráficos. Es el director del máster en Ciencia de datos (Data Science) de la Universitat Oberta de Catalunya (UOC). Sus principales ámbitos de investigación incluyen la inteligencia artificial, el aprendizaje automático, minería de grafos y la privacidad de datos. Desde 2020 es líder de grupo en ADaS Lab (Applied Data Science Lab) donde investiga la aplicación de algoritmos de inteligencia artificial a problemas del mundo real.

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Jordi Casas Roma

Cómo citar este recurso de aprendizaje con el estilo Harvard:

Casas Roma, J. (2023) *Arquitecturas y aplicaciones de las redes neuronales convolucionales*. [Recurso de aprendizaje textual]. 1ª edición. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edición: febrero 2024

© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoría: Jordi Casas Roma

Producción: FUOC

Todos los derechos reservados

Ninguna parte de esta publicación, incluyendo el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma ni por ningún medio, sea este eléctrico, químico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita de los titulares de los derechos.

Índice

Introducción	5
Objetivos	6
1. Arquitecturas de CNN para clasificación	7
1.1. Arquitecturas para clasificación	7
1.1.1. Redes convolucionales clásicas	8
1.1.2. <i>Residual Networks</i> (ResNet)	12
1.1.3. <i>Inception</i>	13
1.2. Arquitecturas para segmentación	16
1.2.1. U-Net	16
2. Transferencia del aprendizaje	20
3. Expansión artificial de datos	23
4. Aplicaciones y ejemplos	27
4.1. Reconocimiento facial	27
4.1.1. <i>Triplet loss</i>	29
4.2. Transferencia de estilos	30
4.2.1. Coste de contenido	31
4.2.2. Coste de estilo	32
5. Consejos prácticos	34
Resumen	36
Glosario	37
Bibliografía	38

Introducción

En este módulo didáctico revisaremos las principales arquitecturas y aplicaciones de las redes neuronales convolucionales (en inglés, *Convolutional Neural Networks*, CNN).

Iniciaremos el texto revisando algunas de las principales arquitecturas de CNN, desde las arquitecturas clásicas que han asentado las bases para el desarrollo de otras arquitecturas más complejas y modernas, hasta otros modelos y tipos de arquitecturas más actuales. En esta capítulo veremos algunas arquitecturas destacadas para clasificación de imágenes, pero también revisaremos arquitecturas que permitan abordar el problema de la segmentación de imágenes. Es importante destacar que este módulo no pretende ser una revisión exhaustiva de arquitecturas de redes convolucionales, si no una revisión de las principales arquitecturas que han inspirado al desarrollo de las arquitecturas actuales, y probablemente futuras, de redes neuronales.

En segundo lugar, revisaremos dos conceptos importantes para el uso de CNN en entornos reales, ya sea en el ámbito empresarial o en el ámbito de la investigación: por un lado, veremos el concepto de la «transferencia del aprendizaje» (en inglés, *transfer learning*) que nos permitirá emplear modelos preentrenados; por otro lado, el aumento artificial de datos (*data augmentation*) que nos permite generar más datos de entrenamiento a partir de nuestros datos disponibles.

El cuarto bloque de este módulo se centrará en algunas aplicaciones relevantes en el ámbito de las CNN, como son el reconocimiento facial y la transferencia de estilo. A continuación, finalizaremos el texto con un breve bloque de consejos prácticos para el desarrollo e implementación de CNN en aplicaciones reales.

Objetivos

En los materiales didácticos de este módulo encontraremos las herramientas indispensables para asimilar los siguientes objetivos:

1. Conocer las principales arquitecturas de redes neuronales convolucionales, tanto en problemas de clasificación como de segmentación de imágenes.
2. Comprender el funcionamiento y las principales ventajas de la transferencia del aprendizaje.
3. Saber cuándo es necesario y cómo aplicar procesos de aumento de datos.
4. Conocer algunas de las principales aplicaciones de las CNN.

1. Arquitecturas de CNN para clasificación

La **clasificación de imágenes**, también llamada reconocimiento de imágenes, tiene como objetivo una clase o etiqueta a una imagen dada a partir de un conjunto finito de clases o etiquetas (es decir, un conjunto de dos o más clases o etiquetas), generalmente teniendo en consideración el contenido de la imagen.

Cuando el conjunto clases o etiquetas contiene únicamente dos ítems, hablamos de clasificación binaria. Por ejemplo, a partir de una imagen (véase figura 1) queremos separar las imágenes que contienen un coche del resto de las imágenes (que no contienen ningún coche).

Figura 1. Ejemplo de problema de clasificación de imágenes, donde queremos diferenciar las imágenes en donde aparece un coche del resto de imágenes (en donde no aparece ningún coche)



En el caso de problemas multiclase (*multi-class*), el concepto y problemática es exactamente el mismo, pero en este caso disponemos de más de dos clases o etiquetas. Continuando con el ejemplo anterior, el objetivo del modelo podría ser indicar si en la imagen aparecen coches, motocicletas, bicis, camiones, autobuses o autocaravanas. Por lo tanto, tenemos un total de seis clases o etiquetas.

A continuación, revisaremos las principales arquitecturas para lidiar con problemas de clasificación.

1.1. Arquitecturas para clasificación

Este capítulo está dedicado a revisar la arquitectura de las redes neuronales más usadas en visión por ordenador. Empezaremos viendo tres de las principales redes convolucionales clásicas en la sección 1.1.1. Continuaremos viendo la estructura y el funcionamiento de las redes residuales (sección 1.1.2) y finalizaremos con un repaso de la red *Inception*, en la sección 1.1.3.

1.1.1. Redes convolucionales clásicas

En esta sección detallaremos la estructura y el funcionamiento de las tres redes convolucionales clásicas, que han inspirado muchos otros modelos actuales: LeNet-5, AlexNet y VGG.

Si se quieren leer los artículos originales se recomienda empezar por AlexNet, seguido de VGG y finalmente LeNet. LeNet es la red más pequeña de las tres, pero el artículo es un poco complicado de leer, por lo tanto, recomendamos dejarlo como última lectura.

Estas redes, además de tener un interés histórico innegable, han servido de inspiración para otras redes más potentes y avanzadas, como la *Residual Network* o *Inception*, que veremos en las secciones siguientes de este capítulo.

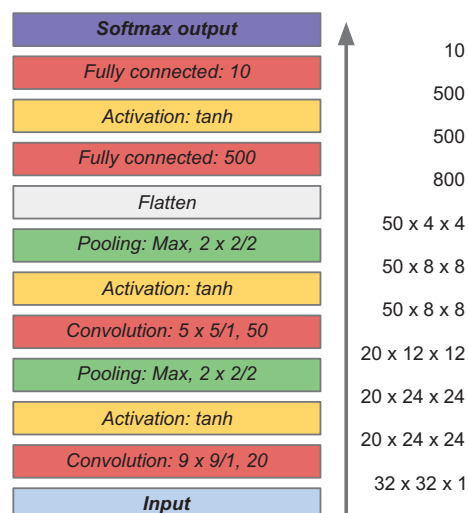
LeNet-5

El objetivo de LeNet-5 (Bengio *et al.*, 1998) era el reconocimiento óptico de caracteres (*Optical Character Recognition*, OCR) y dígitos escritos manualmente en documentos. La arquitectura LeNet-5 es bastante pequeña y, por lo tanto, es un buen inicio para aprender y hacer las primeras pruebas de redes convolucionales. En este caso, dado el tamaño de la red, incluso puede ejecutarse sobre un ordenador que no disponga de GPU, y ejecutar todos los cálculos en la misma CPU.

Lectura complementaria

Y. Bengio, Y. LeCun, L. Bottou, P. Haffner.
«Gradient based learning applied to document recognition». Proc. IEEE, 1998.

Figura 2. Arquitectura de la red LeNet-5



La entrada es una imagen de $32 \times 32 \times 1$. Esta red fue entrenada en imágenes en escala de gris, por esto la dimensión del canal es igual a 1. La figura 2 muestra la arquitectura LeNet-5.

Las cajas marcadas en rojo son capas convolucionales o completamente conectadas (FC). Para cada capa convolucional se muestra el tamaño del filtro ($f \times f$), el *stride* (s) y el número de filtros usados. Para cada capa completamente conectada (FC) se muestra el número de neuronas que la forman. Las capas de *pooling* se muestran en cajas de color verde junto con el tipo de agrupación usado (ya sea *max* o *average*) y el tamaño de filtro y *stride*. Las cajas amarillas muestran el tipo de activación. Es interesante mencionar que cuando se propuso la arquitectura de LeNet-5 las funciones de activación más usadas eran *sigmoid* y *tanh*, mientras que recientemente la función no lineal más usada es la ReLU. La salida o predicción de esta red neuronal será uno de los 10 posibles valores que corresponden a los dígitos de 0 a 9, tal y como hemos visto en las capas de salida *softmax*. Los números de la derecha hacen referencia a las dimensiones de los volúmenes después de hacer cada operación.

Esta red neuronal contiene unos 60.000 parámetros que se deben ajustar durante el proceso de entrenamiento, mientras que actualmente se diseñan arquitecturas de redes neuronales convolucionales con un volumen de cientos o miles de millones de parámetros.

Una característica muy importante y que tienen en común todas las redes neuronales es que a más profundidad, aumenta el número de canales, mientras que el volumen disminuye. Otro patrón que se mantiene es que las capas de convolución van seguidas de una capa de agrupamiento o *pooling*, a continuación una o varias capas completamente conectadas (FC) y, finalmente, la salida de la red.

AlexNet

La red neuronal AlexNet (Krizhevsky *et al.*, 2012) tiene bastantes similitudes con LeNet-5, pero una diferencia importante es que AlexNet es mucho más grande, como se puede apreciar en la figura 3. Mientras que LeNet-5 tenía alrededor de 60.000 parámetros que debían ser ajustados durante el proceso de entrenamiento, AlexNet tiene alrededor de 60 millones de parámetros.

Una consecuencia del tamaño de esta red es que se necesita un volumen de datos bastante grande, dado que durante el entrenamiento se deben ajustar los valores de muchísimas neuronas de las capas ocultas de la red. A cambio, esta red obtiene resultados muy prometedores.

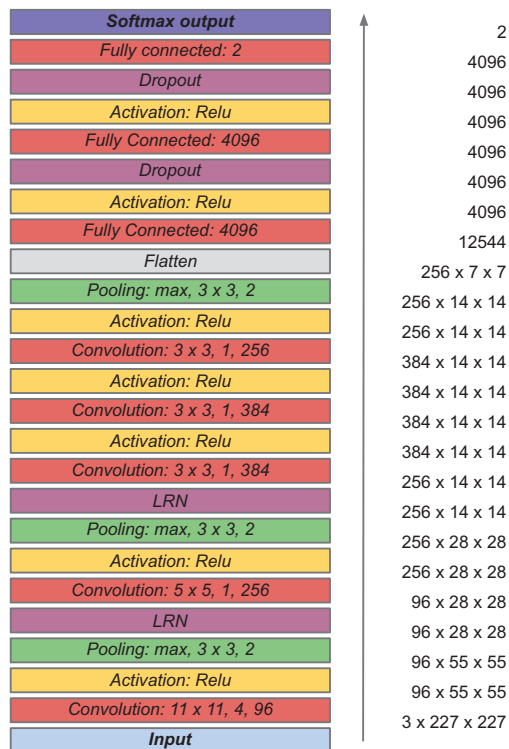
Otro aspecto destacable de esta arquitectura, que le ayuda a mejorar el rendimiento de la red LeNet-5, es el uso de ReLU como función de activación. Esta función de activación es contemporánea a otras funciones clásicas, como la función *sigmoide* o *tanh*, pero ha demostrado ser mejor que la función *tanh*.

Lectura complementaria

A. Krizhevsky, I. Sutskever, G. E. Hinton. «Imagenet classification with deep convolutional neural networks». Advances in Neural Information Processing Systems, Vol. 25, p. 1097-1105, 2012.

Por otra parte, esta arquitectura fue propuesta cuando las GPU todavía eran un poco lentas, el artículo original (Krizhevsky *et al.*, 2012) propone un método de entrenamiento un poco complejo para entrenar la red utilizando dos GPU, con el objetivo de mejorar el tiempo de entrenamiento. La idea es dividir las capas en dos GPU diferentes y estas GPU se podían comunicar entre ellas. Actualmente, este proceso no es necesario, dado el aumento en la capacidad de cálculo de las GPU modernas.

Figura 3. Arquitectura de la red AlexNet



Si observamos la figura 3 veremos que hay una capa llamada LRN (*Local Response Normalization*), que corresponde a las cajas de color morado. Estas capas no se usan demasiado en actualidad, pero vamos a verlas brevemente.

Imaginemos que tenemos un volumen de $13 \times 13 \times 256$, las LRN miran a una posición del volumen y normalizan los valores a través de los 256 canales. La motivación de esta capa es que para cada posición del volumen 13×13 no queremos muchas neuronas con un valor de activación alto.

Antes de que se propusiera la arquitectura AlexNet, el *deep learning* había empezado a ganar terreno en áreas como el reconocimiento del habla, pero fue esta arquitectura la que realmente convenció a la comunidad de visión por ordenador que mediante las redes neuronales convolucionales se podían conseguir resultados superiores a los obtenidos hasta el momento con otros tipos de algoritmos o modelos de aprendizaje automático, como por ejemplo las máquinas de vectores de soporte (*Support-Vector Machines*, SVM).

VGG

La última de las redes clásicas que veremos en este capítulo es la red conocida como VGG (Simonyan & Zisserman, 2014) y de la cual podemos ver la arquitectura en la figura 4.

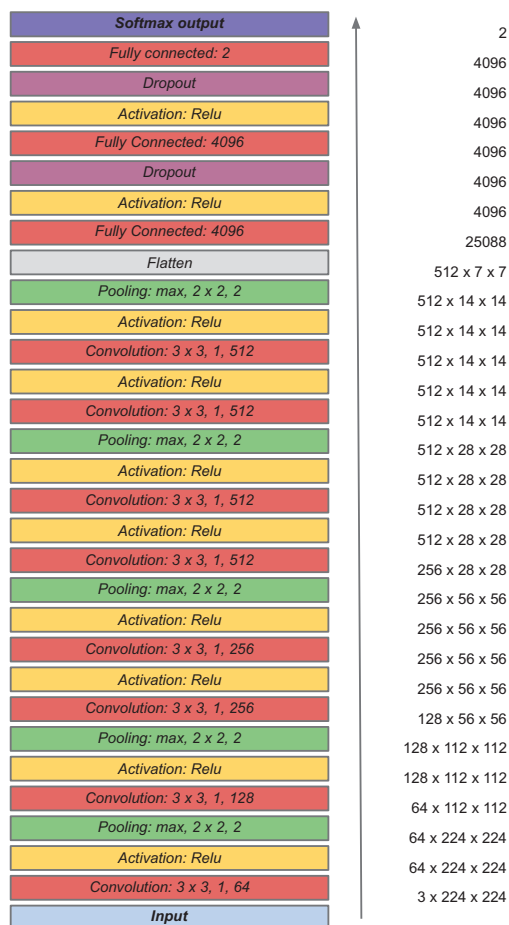
Una de las propiedades más remarcables de esta red neuronal es que en lugar de tener muchos hiperparámetros, sus autores usaron una red muy simple donde se usan solo capas convolucionales con filtros de 3×3 y *stride* $s = 1$, y capas de *pooling* con función de agrupamiento *max-pooling* de 2×2 y *stride* igual a 2 ($s = 2$).

Esta arquitectura es bastante profunda y tiene un total de 138 millones de parámetros, pero su simplicidad la hace muy atractiva. VGG tiene una arquitectura bastante uniforme, las capas convolucionales son seguidas por capas de *pooling* que reducen el alto y ancho de los volúmenes. Además, si nos fijamos en el número de filtros que se usan en cada convolución, sus valores son: 64, 128, 256 y 512. Es decir, el valor se duplica en cada paso, creando un principio bastante simple en el diseño de la arquitectura.

Lectura complementaria

K. Simonyan, A. Zisserman.
«Very deep convolutional
networks for large-scale
image recognition». CoRR,
abs/1409.1556, 2014.

Figura 4. Arquitectura de la red VGG



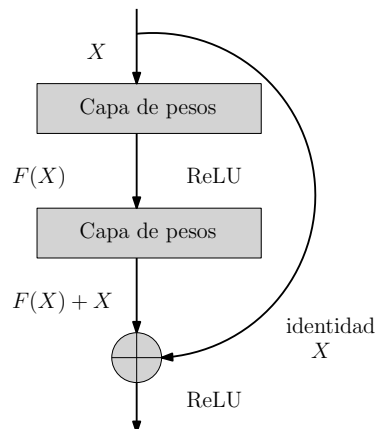
Sin embargo, la principal desventaja de esta red es el gran número de parámetros que contiene y que hay que ajustar durante el proceso de entrenamiento de la red.

En la literatura encontramos referencias a las redes VGG-16 y VGG-19. VGG-19 es una versión todavía más profunda que VGG-16, pero la precisión obtenida por las dos es muy similar y normalmente se usa la red VGG-16.

1.1.2. Residual Networks (ResNet)

Los autores de las redes residuales (*Residual Networks*, ResNet) (He *et al.*, 2016) observaron que la bondad de una red no debería ser estrictamente dependiente de la profundidad de la misma. Esto se debe a que si sostenemos que la red neuronal puede aproximar cualquier función compleja, también podría aprender la función de identidad, es decir, entrada = salida, saltando de manera efectiva el progreso de aprendizaje en algunas capas. Pero, en el mundo real, este no es el caso debido al problema de la desaparición y explosión del gradiente (*vanishing/exploding gradient*) (Szegedy *et al.*, 2017).

Figura 5. Residual ResNet



Por lo tanto, podría ser útil forzar explícitamente a la red a aprender una asignación de identidad, aprendiendo el residuo de entrada y salida de algunas capas (o subredes), como se muestra en la figura 5. Supongamos que la entrada de la subred es X y el verdadero resultado es $H(X)$. El residual es la diferencia entre ellos:

$$F(X) = H(X) - X \quad (1)$$

Como estamos interesados en encontrar el verdadero resultado subyacente de la subred, reorganizamos esta ecuación, que queda de la siguiente forma:

$$H(X) = F(X) + X \quad (2)$$

Lectura complementaria

K. He, X. Zhang, S. Ren y J. Sun. «Deep Residual Learning for Image Recognition», 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, p. 770-778, doi: 10.1109/CVPR.2016.90.

Lectura complementaria

C. Szegedy, S. Ioffe, V. Vanhoucke y A. A. Alemi. «Inception-v4, inception-ResNet and the impact of residual connections on learning». Proc. of Conference on Artificial Intelligence (AAAI'17). AAAI Press, p. 4278-4284, 2017.

Esta es la principal diferencia entre ResNet y las redes neuronales tradicionales. Donde las redes neuronales tradicionales aprenderán $H(X)$ directamente, ResNet modela las capas para conocer el residuo de entrada y salida de las subredes. Esto le dará a la red una opción para omitir subredes haciendo $F(X) = 0$ y $H(X) = X$. En otras palabras, la salida de una subred en particular es solo la salida de la última subred.

Durante la retropropagación (*backpropagation*), el aprendizaje residual nos da una buena propiedad. Debido a la formulación, la red podría optar por ignorar el gradiente de algunas subredes y simplemente reenviar el gradiente desde capas superiores a capas inferiores sin ninguna modificación. Como ejemplo extremo, esto significa que ResNet podría reenviar el gradiente desde la última capa directamente a la primera capa. Esto le da a ResNet una opción adicional que puede ser útil, en lugar de simplemente hacer cálculos en todas las capas.

1.1.3. Inception

La inspiración de esta red proviene de un problema de decisión que se plantean los autores en el momento de diseñar la arquitectura de una red convolucional: ¿qué tipo de convolución aplicamos en cada capa?, ¿usaremos un filtro de 3×3 ?, ¿o un filtro de 5×5 ?

La solución a esta cuestión es el fundamento de esta red: ¿y porque no usarlos todos y hacer que sea el propio modelo quien decida? En este sentido, *Inception* (Szegedy *et al.*, 2016) hace todas las convoluciones de forma paralela y concatena los resultados en mapas de características antes de ir a la siguiente capa.

Supongamos que la siguiente capa también es un módulo *Inception*. Cada uno de los mapas de características de las convoluciones pasará como un conjunto mixto de convoluciones a la siguiente capa. La idea es que no necesitamos decidir qué tipo de convoluciones es mejor, 3×3 o 5×5 . En su lugar, hacemos todas las convoluciones y dejamos que sea el modelo que escoja la mejor. Además, esta arquitectura permite al modelo recuperar tanto características locales mediante convoluciones pequeñas como características de alto nivel mediante convoluciones más grandes. Ahora que tenemos la idea básica, vamos a ver la arquitectura específica.

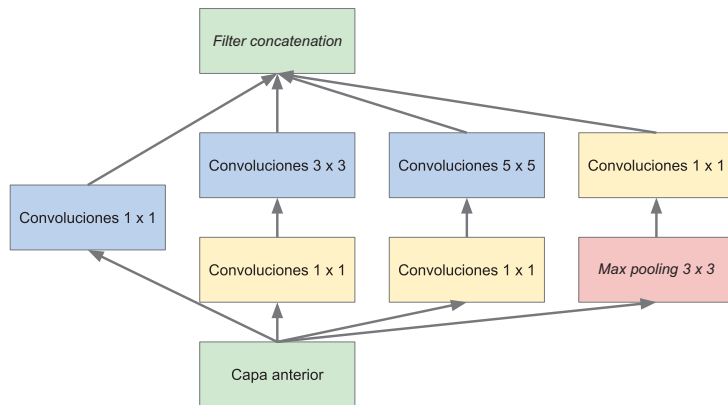
La figura 6 muestra la arquitectura de un solo módulo *Inception*. En este caso específico, estamos usando convoluciones 1×1 , 3×3 y 5×5 junto con un agrupamiento basado en la función de *max-pooling* de 3×3 . El *pooling* se añade al módulo *Inception* porque todas las redes neuronales que funcionan bien tienen esta función (esto se ha probado de forma experimental en artículos de referencia). Las convoluciones más grandes tienen un coste computacional y temporal mayor. Por este motivo, se suelen realizar primero las convoluciones

Lectura complementaria

C. Szegedy, S. Ioffe, V. Vanhoucke. «Inception-v4, inception-resnet and the impact of residual connections on learning». CoRR, abs/1602.07261, 2016.

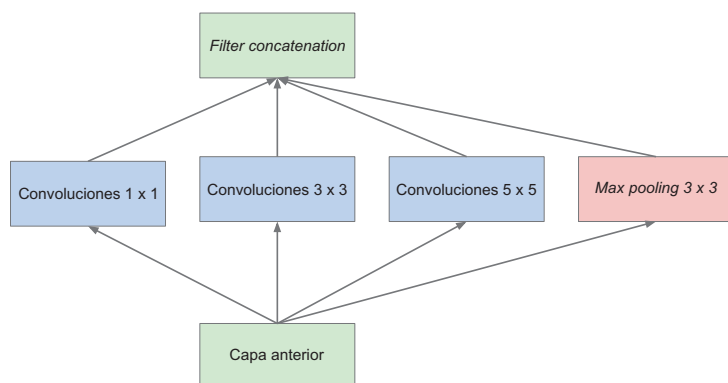
1×1 , reduciendo de esta forma la dimensionalidad del mapa de características, pasar el resultado a través de una ReLU y, finalmente, hacer las convoluciones más grandes (en este caso 3×3 y 5×5). Por lo tanto, es importante explicitar que la convolución 1×1 es clave en este tipo de redes, ya que permite reducir la dimensionalidad del mapa de características.

Figura 6. Módulo *Inception*



Para ver porqué el uso de las convoluciones 1×1 para reducir la dimensionalidad de la entrada permite que los cálculos sean más razonables, vamos primero a ver porqué necesitamos afrontar un problema computacional muy costoso si no usamos este método de reducción de dimensionalidad.

Figura 7. Módulo *Inception naïve*

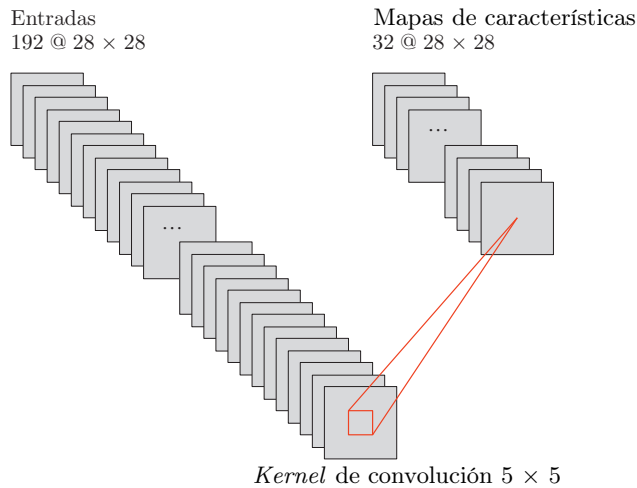


Supongamos que usamos una implementación *naïve* del módulo *Inception*. La figura 7 muestra el módulo, que es muy similar al de la figura 6, pero en este caso no tenemos las convoluciones adicionales 1×1 antes de las convoluciones más grandes (3×3 y 5×5). Vamos a examinar el número de cálculos necesarios para el primer módulo de la red *Inception*.

Para simplificar un poco los cálculos, podemos asumir que la red usa el mismo *padding* para las convoluciones del mismo módulo, dado que la entrada y la salida tienen una dimensión de 28×28 . Vamos a ver el coste de las convoluciones 5×5 si no aplicamos previamente la reducción de la dimensionalidad.

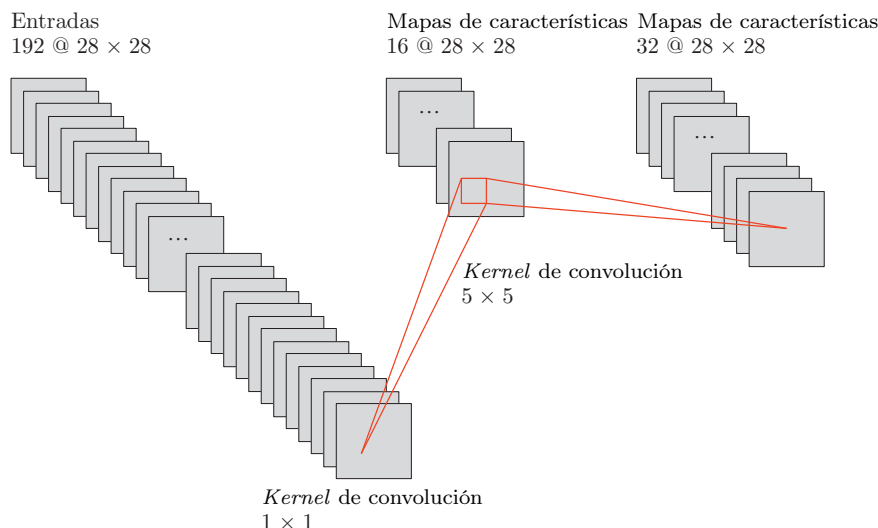
La figura 8 muestra de forma gráfica estas operaciones. Como se puede verificar, estamos hablando de $5^2 \times 28^2 \times 192 \times 32 = 120.422.400$ operaciones... y ¡son muchos cálculos! Ahora podemos entender el porqué los autores de esta red necesitan intentar, de alguna forma, reducir este número.

Figura 8. Convoluciones del módulo *inception* usando la versión *naïve*



Para tal fin, usamos el modelo de la figura 6. Previamente a realizar las convoluciones 5×5 , aplicamos una capa anterior con una convolución de 1×1 , que devuelve como salida un mapa de características de 16 @ 28 × 28. A continuación, aplicamos la convolución 5×5 en estos mapas de características que dan una salida de 32 @ 28 × 28. Este proceso se muestra en la figura 9.

Figura 9. Convoluciones 1×1 que reducen de la dimensionalidad con el objetivo de reducir el número de operaciones en las convoluciones 5×5



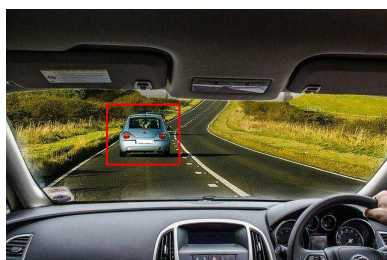
En este caso habrá $(1^2 \times 28^2 \times 192 \times 16) + (5^2 \times 28^2 \times 16 \times 32) = 12.443.648$ operaciones. Aunque sea un número todavía grande, se reduce el número de cálculos con respecto al modelo *naïve* con un factor de 10. Queda como ejercicio para el lector realizar los cálculos para las convoluciones de 3×3 , que siguen el mismo proceso.

1.2. Arquitecturas para segmentación

La **segmentación de imágenes** tiene como objetivo indicar la posición de un determinado objeto dentro de una imagen dada, en el caso de existir.

Un ejemplo representativo es la segmentación de imágenes médicas, en donde se entrenan modelos predictivos para segmentar los diferentes tejidos de una imagen de rayos X o imagen de resonancia magnética, entre otras. Otro ejemplo ilustrativo, en este caso en el campo de la conducción autónoma, está relacionado con identificar la posición de otros vehículos dentro de una imagen. Es decir, no es suficiente que un modelo nos indique si hay otro vehículo en nuestro campo de visión, sino que debemos saber la posición exacta de este dentro de la imagen para poder evitar una colisión, como se puede ver en la figura 10.

Figura 10. Ejemplo de problema de segmentación de imágenes, donde queremos identificar la posición de un determinado objeto dentro de la imagen



A diferencia de los modelos de clasificación, que ofrecen una salida basada en el número de clases o etiquetas del conjunto de datos, los modelos de segmentación deben indicar la posición en la imagen de un determinado objeto o característica de la imagen. En este sentido, las arquitecturas vistas en las secciones anteriores no permiten, por lo general, reportar este tipo de información. Por lo tanto, las redes convolucionales para problemas de segmentación presentan una arquitectura sensiblemente diferente a las vistas anteriormente.

A continuación, revisaremos una de las principales arquitecturas de CNN para problemas de segmentación, que ha inspirado muchas otras redes derivadas. Al igual que en el capítulo anterior, una revisión exhaustiva de las principales redes relevantes de este ámbito queda fuera de alcance de este texto.

1.2.1. U-Net

U-Net (Ronneberger *et al.*, 2015) es un tipo de arquitectura de red neuronal convolucional que se usa comúnmente en tareas de segmentación de imágenes, **especialmente diseñada para la segmentación de imágenes biomédicas**. La arquitectura consta de un codificador (fase de contracción) y un decodificador (fase de expansión), conectados por varias conexiones de salto que concatenan características del codificador y el decodificador.

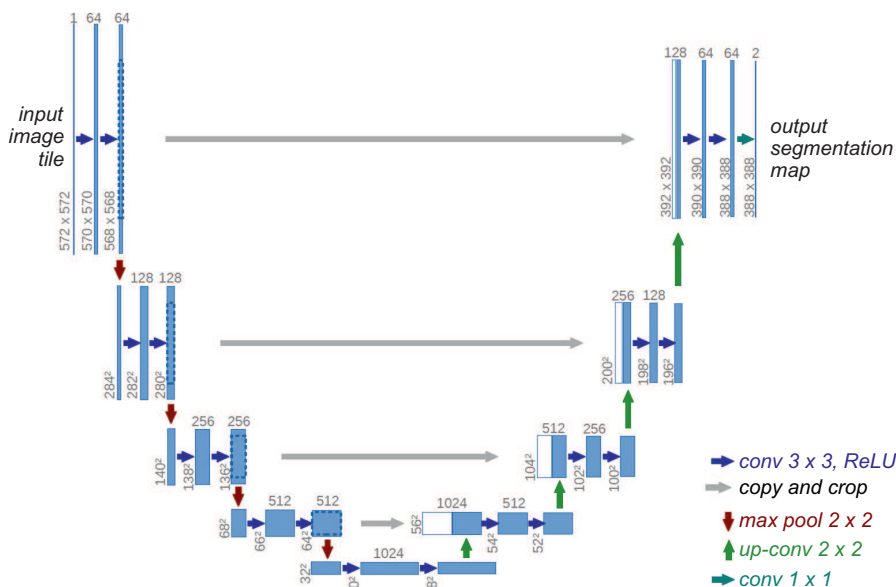
Lectura complementaria

O. Ronneberger, P. Fischer, T. Brox (2015). «U-Net: Convolutional Networks for Biomedical Image Segmentation». Proc. MICCAI 2015. Lecture Notes in Computer Science, Vol. 9351. Springer, Cham. doi: 10.1007/978-3-319-24574-4_28.

La parte del **codificador** de la red consta de múltiples capas convolucionales y de agrupación que extraen características de la imagen de entrada y reducen su resolución espacial. A continuación, la parte del **decodificador** usa capas de muestreo ascendente para aumentar la resolución espacial de las características y reconstruir la imagen de salida.

Las **conexiones de salto** entre el codificador y el decodificador permiten que la red retenga información detallada importante de la imagen de entrada durante todo el proceso de codificación y decodificación. Esto hace que U-Net sea ideal para tareas de segmentación de imágenes, donde es importante preservar los detalles de la imagen de entrada en el mapa de segmentación de salida.

Figura 11. Arquitectura de la red U-Net (ejemplo para 32x32 píxeles en la resolución más baja). Cada cuadro azul corresponde a un mapa de funciones multicanal. El número de canales se indica en la parte superior de la caja. El tamaño x-y se proporciona en el borde inferior izquierdo del cuadro. Los cuadros blancos representan mapas de características copiados. Las flechas indican las diferentes operaciones.



Fuente: (Ronneberger *et al.*, 2015).

La figura 11 muestra el detalle de la arquitectura U-Net, donde podemos ver la imagen de entrada en la parte superior izquierda de la figura (en este caso concreto, con una resolución de 572×572 píxeles y un único canal de profundidad). Los datos son propagados por todos los caminos posibles, es decir, haciendo el recorrido completo de contracción y expansión (lo que le proporciona la típica forma de «U») y, también, por los canales de salto que conectan directamente las capas del codificador y decodificador, como se muestra en el esquema. La salida del modelo, es decir, la segmentación de la imagen, se puede observar en la parte superior derecha.

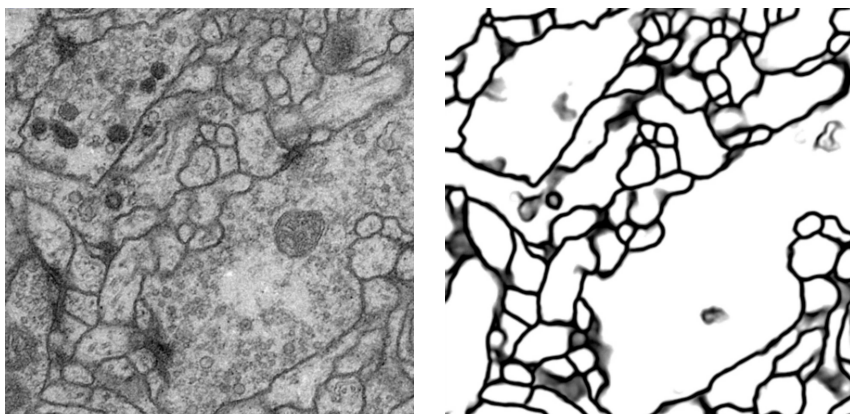
La primera parte de la red, a la que llamamos **codificador** y que realiza la función de contracción, consiste en múltiples capas convolucionales seguidas de

capas de activación no lineal ReLU. Estas capas funcionan de la misma forma que las redes vistas anteriormente. En la figura 11 se muestran mediante la flecha azul oscuro, empleando siempre un *kernel* de 3×3 . A continuación, vemos que se aplica una función de agrupación, en este caso concreto de *max-pooling* con un valor de paso igual a 2 ($s = 2$), lo que permite reducir a la mitad la dimensión del mapa de características anterior. Al igual que las redes vistas anteriormente, las dimensiones de la imagen se van reduciendo, mientras que el número de canales aumenta, durante el proceso de contracción.

En la segunda parte de la red, que hemos llamado decodificador y que realiza la función de expansión, se crea un mapa de segmentación de alta resolución. Este proceso se basa en una secuencia de *up-convolutions* y concatenaciones con los mapas de características de alta resolución que provienen de las conexiones de salto (*contracting paths*).

En la capa final, se usa una convolución de 1×1 para asignar cada vector de características de 64 componentes al número deseado de clases, que en este caso concreto, como se puede apreciar en la figura comentada, es de 2 clases. En total la red tiene 23 capas convolucionales.

Figura 12. Ejemplo de segmentación de una imagen



(a) Imagen original

(b) Segmentación

Fuente: (Ronneberger *et al.*, 2015).

La figura 12 muestra un ejemplo de segmentación empleando la arquitectura de red U-Net que hemos comentado anteriormente, donde se puede ver la imagen original (que corresponde a la estructura celular del cerebro) en la parte izquierda, y el resultado de la segmentación en la parte derecha de la figura.

Una de las principales ventajas de la arquitectura U-Net es que requiere de un volumen reducido de imágenes anotadas (etiquetadas) para su entrenamiento, que la hacen especialmente relevante en dominios donde es habitual entrenar modelos con pocos datos etiquetados o anotados, como suelen ser los datos médicos.

Lectura complementaria

N. Siddique, S. Paheding, C. P. Elkin y V. Devabhaktuni, «U-Net and Its Variants for Medical Image Segmentation: A Review of Theory and Applications». IEEE Access, vol. 9, p. 82031-82057, 2021, doi: 10.1109/AC-CESS.2021.3086020.

En general, las arquitecturas U-Net⁽¹⁾ son populares para tareas como la segmentación de imágenes médicas, la segmentación de imágenes de satélite y la sustracción de fondo en transmisiones de vídeo, entre otras aplicaciones relevantes.

⁽¹⁾<https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>

A partir del éxito de esta primera arquitectura U-Net, se han desarrollado múltiples arquitecturas basadas en el mismo concepto, generalmente añadiendo más capas y complejidad al modelo inicial. Una revisión exhaustiva queda fuera del alcance de este texto.

2. Transferencia del aprendizaje

La **transferencia del aprendizaje** (o *transfer learning*) es una técnica de aprendizaje automático que consiste en **transferir un modelo que ha sido entrenado para una tarea determinada para ser usado y resolver otra tarea parecida.**

Esta optimización permite progresar de forma más rápida y mejorar el rendimiento del modelo. Las técnicas de *transfer learning* son muy populares en el área de *deep learning*, debido a la gran cantidad de datos y recursos computacionales que se requieren para entrenar un modelo.

Sin embargo, es importante destacar que **esta técnica solo funciona si las características que ha aprendido el modelo durante el entrenamiento de la primera tarea son lo suficientemente generales para ser útiles y aplicables en otros entornos similares.** Esta forma de transferir el aprendizaje que se usa en *deep learning* se llama **transferencia inductiva** (*inductive transfer*).

En general, un modelo se puede ajustar de forma beneficiosa a una tarea diferente, siempre que esta esté relacionada con la tarea original. Como hemos visto anteriormente, **el entrenamiento de las redes neuronales consiste, en esencia, en ajustar los pesos de la red empleando los datos etiquetados del conjunto de datos.** Estos pesos, una vez «aprendidos», se pueden exportar y transferir a otra red neuronal, en lugar de entrenar los pesos de la red desde cero.

Hay diferentes maneras de usar el *transfer learning*, siendo la más común usar un modelo preentrenado. Un modelo preentrenado es un modelo creado por una tercera persona que resuelve un problema similar al que queremos resolver nosotros. En lugar de construir un modelo desde cero vamos a usar el modelo preentrenado como punto de partida. El funcionamiento es el siguiente:

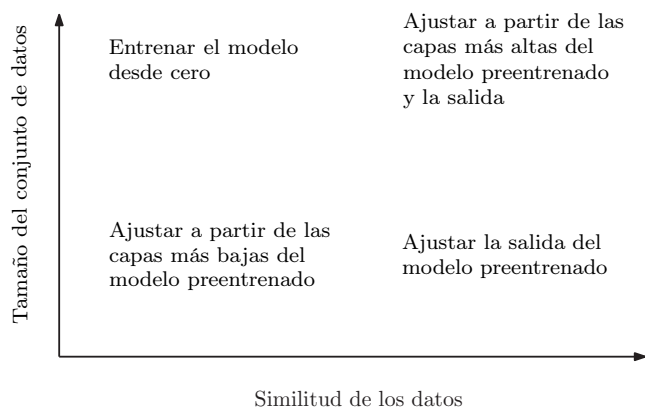
- 1) **Seleccionar el modelo fuente a partir de los modelos disponibles.** Muchas instituciones de investigación ponen a disposición de terceros modelos ya entrenados a partir de grandes volúmenes de datos.
- 2) **Reusar el modelo.** El modelo seleccionado puede ser usado como punto de partida para resolver la tarea de interés. Esto puede implicar usar todas o algunas de las partes del modelo original, dependiendo de la técnica de modelado usada.

3) Ajuste (*tuning*) del modelo. Opcionalmente, el modelo puede necesitar ser adaptado o refinado según los datos requeridos para resolver la tarea de interés.

El proceso de ajuste (habitualmente conocido como *tuning*) del modelo suele ser complejo y bastante costoso. Existen múltiples formas de realizar este proceso, siendo algunas de las más habituales, las siguientes:

- **Extracción de características.** Podemos usar el modelo preentrenado como un método de selección de características. Si borramos la última capa de la red neuronal (la salida que nos da las probabilidades de una instancia de pertenecer a cada una de las clases) y usamos la red entera como un extractor de características fijo para los datos de la nueva tarea.
- **Usar la arquitectura del modelo preentrenado.** Usamos la arquitectura del modelo e inicializamos los pesos de forma aleatoria para entrenar el modelo de nuevo con un nuevo conjunto de datos.
- **Entrenar algunas capas y congelar las otras.** Esto se refiere a reentrenar el modelo de forma parcial. Para ello mantenemos los pesos de las capas iniciales (es decir, «congelamos» estos pesos) y reentrenamos solamente los pesos de las capas más altas.

Figura 13. Diagrama de casos de uso de *transfer learning*



El diagrama de la figura 13 puede servir de ayuda a la hora de decidir qué forma de ajuste es mejor según el volumen de datos disponible y la similitud entre las tareas a realizar. En ella se muestran cuatro escenarios diferentes, que pasamos a comentar de forma individual:

1) El tamaño del conjunto de datos es pequeño, pero la similitud de los datos es muy alta. En este caso, debido a que la similitud de los datos es muy alta, no necesitamos reentrenar el modelo de nuevo. Lo que tenemos que hacer es personalizar y modificar la salida para adaptarla al nuevo problema. Se usa el modelo preentrenado como un extractor de características.

Imaginemos que queremos usar un modelo entrenado en ImageNet para identificar un nuevo conjunto de datos que contiene, solamente, imágenes de perros y gatos. Las imágenes de los dos conjuntos son muy parecidas, no obstante ahora solo tenemos dos categorías en la salida. Por lo tanto, lo que haremos es modificar la capa completamente conectada (FC) y la capa *softmax* para que tenga únicamente dos categorías (o clases) en lugar de las 1.000 del conjunto de datos original.

2) El tamaño de los datos es pequeño y la similitud de los datos es muy baja. En este caso podemos congelar los pesos de las k capas iniciales del modelo preentrenado y entrenar las siguientes capas ($n - k$). Las capas más altas serán personalizadas y adaptadas al nuevo conjunto de datos. Como el nuevo conjunto de datos tiene una muy baja similitud, es importante ajustar las capas altas según los datos contenidos en el nuevo conjunto.

El tamaño pequeño del conjunto de datos se puede ver compensado por el hecho que las capas iniciales mantienen los pesos del modelo inicial, que ha sido entrenado utilizando un conjunto de datos mucho más grande.

3) El tamaño del conjunto de datos es grande, pero la similitud de los datos es baja. En este caso, debido a que tenemos un conjunto de datos grande, el entrenamiento de la red neuronal será efectivo. No obstante debido a que los datos son diferentes comparados con los datos iniciales que se usaron para entrenar la red, los pesos aprendidos quizás no sean del todo óptimos y es mejor entrenar la red desde cero.

4) El tamaño del conjunto de datos es grande y la similitud de los datos es alta. Esta es la situación ideal. En este caso el modelo preentrenado debería de ser muy efectivo. La mejor manera de usarlo es reentrenar la arquitectura del modelo y los pesos iniciales de este. Entonces podemos reentrenar el modelo usando los pesos del modelo preentrenado como método de inicialización.

3. Expansión artificial de datos

Existe una relación casi lineal entre la cantidad de datos que se requieren para entrenar un modelo y el tamaño del modelo. El modelo debe de ser lo suficientemente grande para capturar las relaciones existentes en los datos (texturas, formas, etc.) junto con las especificidades del problema (e.g. número de categorías o clases).

Las capas más iniciales del modelo capturan relaciones de alto nivel entre las diferentes partes de la entrada, tales como aristas o patrones. Las capas posteriores capturan información que ayuda al modelo a tomar la decisión final. Es decir, la información que ayuda a discriminar entre las diferentes salidas que puede generar del modelo. Si el problema es complejo, como la clasificación de imágenes, el número de parámetros y la cantidad de datos requeridos es muy alto. Por lo tanto, en estos casos, disponer de un conjunto de datos grande (o muy grande) es crucial para obtener buenos resultados en *deep learning*. Sin embargo, no siempre es posible disponer de grandes conjuntos de datos etiquetados y estos se han convertido en uno de los grandes problemas del *deep learning* y del aprendizaje automático en general.

¿Por qué es importante tener muchos datos?

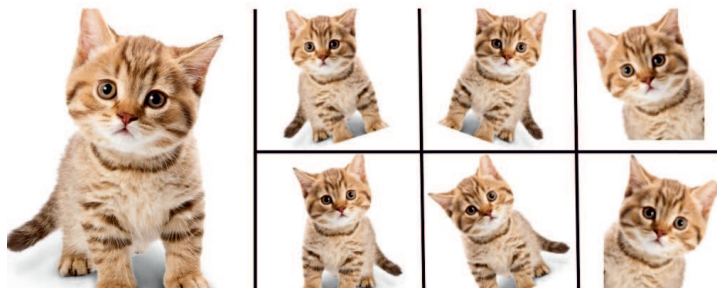
Cuando entrenamos un modelo de *deep learning*, lo que realmente estamos haciendo es ajustar los parámetros para que puedan expresar la relación entre la entrada (una imagen) y la salida (una clase) de la red. Las arquitecturas de los modelos empleados en el estado del arte de *deep learning* tienen un orden de millones de parámetros que se deben entrenar y ajustar. En consecuencia, se necesitan proporcionar muchos ejemplos al modelo para que sea capaz de aprender los mejores parámetros y obtener un buen rendimiento.

¿Cómo obtenemos más datos si nuestro conjunto es pequeño?

Una opción ampliamente utilizada para obtener más datos de los inicialmente disponibles consiste en hacer alteraciones en las imágenes existentes de nuestro conjunto de datos. Mediante cambios pequeños en las imágenes originales, como por ejemplo traslaciones, rotaciones o volteos (*flip*), obtendremos más imágenes que podemos emplear para entrenar la red, ya que la red neuronal las interpretará como imágenes distintas.

La figura 14 muestra como con pequeñas alteraciones en la imagen original es posible obtener un conjunto de imágenes nuevas.

Figura 14. Alteraciones de la imagen original mediante técnicas de expansión artificial de datos



Fuente: <https://medium.com>

Las CNN son invariantes a traslaciones, punto de vista, tamaño e iluminación. Esta es la premisa de la expansión artificial de datos (o *data augmentation*). Es decir, generamos imágenes de forma artificial (sintéticamente) para obtener datos en una variedad de condiciones que nos acerque a la realidad que nos encontraremos a la hora de resolver el problema en el mundo real.

Esta práctica no se usa solamente cuando tenemos conjuntos de datos pequeños, sino que se ha demostrado que aunque tengamos un conjunto de datos suficientemente grande, ayuda a tener más datos relevantes para el entrenamiento del modelo. La red neuronal será tan buena (o mala) como los son los datos que utilizamos para entrenarla.

Antes de empezar con las diferentes técnicas de *data augmentation*, vamos a ver que hay dos técnicas principales de aplicarla:

- **Offline augmentation.** Todas las transformaciones se hacen antes de entrenar el modelo, aumentando de esta forma el tamaño del conjunto de datos. Esta forma de trabajar es la recomendada si tenemos conjuntos de datos pequeños, ya que el tamaño del *dataset* se incrementará proporcionalmente al número de transformaciones que realicemos.
- **Online augmentation** o **augmentation on the fly.** Las transformaciones se hacen en los *mini-batch*, antes de alimentar el modelo. Este método es preferible cuando tenemos conjuntos de datos grandes, dado que no se puede afrontar el incremento de imágenes desde el inicio.

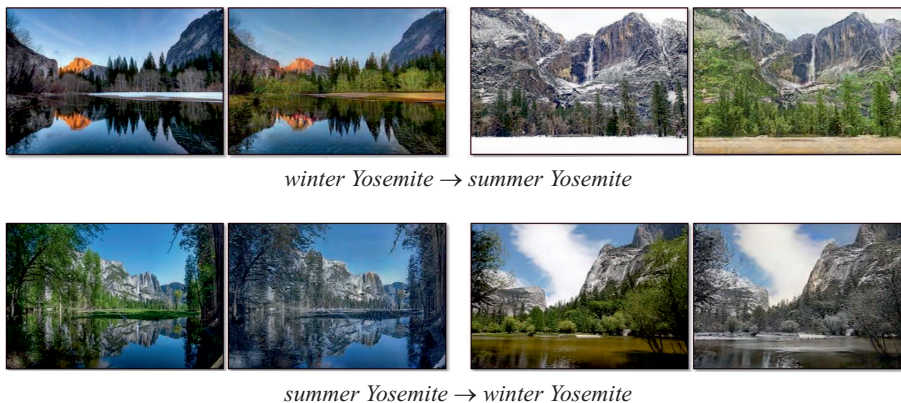
Ya sea de forma *offline* u *online*, las técnicas más populares para expandir o aumentar artificialmente el conjunto de datos son:

- **Dar la vuelta (flip).** A las imágenes se les puede dar la vuelta vertical o horizontalmente. Algunos *frameworks* no tienen la función de dar la vuelta

vertical, pero esto es equivalente a rotar la imagen 180 grados y luego dar la vuelta de forma horizontal.

- **Rotación.** Hay que tener en cuenta que después de realizar esta operación las dimensiones de la imagen se pueden ver alteradas. Si la imagen es cuadrada las dimensiones se mantendrán, pero si es un rectángulo solo se mantendrá si la rotación es de 180 grados.
- **Escalado.** La imagen se puede escalar haciéndola más grande o más pequeña.
- **Recortar (*crop*).** De forma aleatoria seleccionamos una parte de la imagen original y la cortamos. Posteriormente se redimensiona la imagen para que mantenga el tamaño de la imagen original.
- **Traslación.** Esta operación se refiere a desplazar la imagen a lo largo del eje vertical u horizontal. Este método es muy útil porque los objetos se pueden encontrar en casi cualquier parte de la imagen y fuerza a las redes a aprender y buscar los objetos en cualquier parte de la imagen.
- **Añadir ruido.** El *overfitting* normalmente se produce cuando la red neuronal intenta aprender características que se dan con mucha frecuencia. El ruido Gaussiano (mediana 0) genera puntos de datos en todas las frecuencias, distorsionando de esta forma las características más usuales.

Figura 15. Cambio de estaciones usando CycleGAN



Fuente: <https://www.kdnuggets.com>

En el mundo real, las imágenes naturales existen en una variedad de condiciones que no se pueden reproducir con los métodos anteriormente comentados. Por ejemplo, imaginemos la tarea de identificar un paisaje en una fotografía. El paisaje puede ser cualquier cosa: bosques, montañas, nieve, etc. Parece una tarea sencilla si no tenemos en cuenta las diferentes estaciones del año. Si la red neuronal no aprende el hecho de que ciertos paisajes pueden existir en diferentes condiciones (nieve, humedad, sol, etc.), podría llegar a clasificar paisajes nevados como glaciares.

Lectura complementaria

J.-Y. Zhu, T. Park, P. Isola, A. A. Efros. «Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks». 2017 IEEE International Conference on Computer Vision (ICCV), 2017, p. 2242-2251, doi: 10.1109/ICCV.2017.244.

Figura 16. Transferencia de estilos usando *deep learning* para aumentar los datos de un conjunto de datos



En este tipo de casos, se emplean métodos como el *GAN convolutional* (Zhu *et al.*, 2017) (figura 15) u otras redes neuronales de transferencia de estilos para generar nuevas imágenes (Luan *et al.*, 2017) (figura 16).

Lectura complementaria

F. Luan, S. Paris, E. Shechtman, K. Bala. «Deep Photo Style Transfer». 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, p. 6997-7005, doi: 10.1109/CVPR.2017.740.

4. Aplicaciones y ejemplos

En esta parte de este capítulo nos centraremos en discutir un par de ejemplos relevantes en el contexto del *deep learning* y de la visión por ordenador. En concreto, veremos un caso práctico de reconocimiento facial y otro de transferencia de estilo entre imágenes.

4.1. Reconocimiento facial

En esta sección vamos a ver un ejemplo de como resolver el problema de reconocimiento facial mediante redes neuronales convolucionales.

Vamos a empezar primero con la terminología usada en el reconocimiento facial. En la literatura, los autores hablan a menudo de verificación facial y reconocimiento facial. La **verificación facial** consiste en, dados una imagen de entrada y una identificación de una persona (ya sea un nombre, ID o cualquier identificación única), el sistema debe determinar si la imagen de entrada se corresponde con la persona indicada. Este proceso se conoce como un problema *one-to-one*, donde solo nos interesa saber si la persona se corresponde con quien dice que es. El problema del **reconocimiento facial** consiste en, a partir de una imagen de entrada, determinar la identificación de la persona en cuestión (presuntamente, dentro de un conjunto finito de personas conocidas).

Una de las principales problemáticas o complejidades relacionadas con un sistema de verificación se debe, precisamente, a que necesitamos resolver un problema de aprendizaje singular. Es decir, necesitamos reconocer a una persona usando solo una imagen o un solo ejemplo de la cara de la persona. Como ya hemos visto y comentado, los algoritmos de *deep learning* no funcionan demasiado bien con solo una imagen de entrenamiento por cada persona.

Supongamos que tenemos una base de datos de imágenes de empleados de una organización. Uno de ellos aparece en la oficina y quiere que el sistema de identificación lo deje pasar. Aunque el sistema haya visto solamente una imagen del empleado, este debe ser capaz de reconocer positivamente a la persona y permitirle el acceso a las instalaciones. Por el contrario, si aparece una persona que no está en la base de datos, el sistema debe ser capaz de negar el acceso advirtiendo que esta persona no es un empleado de la organización. Esto es así para la mayoría de aplicaciones de reconocimiento facial

reales, porque normalmente solo tenemos en el sistema una sola imagen de los empleados o las personas que queremos reconocer.

Una primera aproximación sería pasar la imagen de la persona por una CNN y dejar que esta nos dé una etiqueta de salida usando una unidad *softmax* con un número de salidas igual al número de empleados. Pero esto realmente no funciona, debido principalmente al número reducido de imágenes de entrenamiento de que disponemos, y porque cada vez que alguien nuevo se uniera a la organización deberíamos de reentrenar el sistema.

Una segunda aproximación más eficaz consiste en aprender una función de «similitud». En particular, queremos una red neuronal que aprenda una función $s(x_i, x_j)$ que dadas dos imágenes de entrada nos indique el grado de diferencia entre las dos imágenes. En particular, si las dos imágenes son la misma persona, queremos que la salida sea un número pequeño y, si las imágenes son de diferentes personas, queremos que la salida sea un número más grande. Es decir, queremos que la red nos devuelva una métrica de la divergencia entre dos imágenes.

Durante el proceso de reconocimiento, si el grado de diferencia es más pequeño que un cierto umbral τ (*threshold*, que es un hiperparámetro) entonces la predicción será que las dos imágenes son de la misma persona y, si el valor devuelto por la red es mayor que τ , entonces la predicción será que son personas distintas.

Así pues, el proceso es el siguiente: dada una imagen x_i , usamos la función $s(x_i, x_j)$ para compararla con cada una de las imágenes que tenemos en nuestra base de datos, i.e. $x_j \in \{x_1, \dots, x_p\} \setminus x_i$. Si la función nos da un número pequeño, i.e. $s < \tau$, entonces determinamos que se trata de la misma persona. Si finalmente ninguna de las comparaciones nos da un valor inferior a τ es porque la persona de la imagen de entrada no se corresponde con ninguna de las que tenemos en la base de datos y, por lo tanto, no la podemos reconocer.

Para entrenar una red neuronal que aprenda la función s se usan lo que conocemos como las **redes siamesas.**

Tenemos una imagen de entrada x_1 y la pasamos a través de la red convolucional terminando con una capa completamente conectada (FC) que finaliza con un vector de características. Normalmente este vector de características es usado como entrada de la función *softmax* para hacer la clasificación. Para el reconocimiento facial nos vamos a focalizar en el vector de características y no en la salida del clasificador. Vamos a llamar a este vector $f(x_1)$, representa una codificación de x_1 .

La forma de construir el sistema de reconocimiento facial es pasar una segunda imagen x_2 a través de la misma red neuronal con los mismos parámetros

obteniendo un vector de salida diferente, que representará la codificación de esta segunda imagen $f(x_2)$.

Finalmente, debemos escoger una función que nos permite calcular la distancia d entre los vectores x_1 y x_2 . Para tal propósito, podemos emplear, por ejemplo, la norma de la diferencia de ambos vectores:

$$d = \|f(x_1) - f(x_2)\|^2. \quad (3)$$

Si $d < \text{tau}$, entonces x_1 y x_2 son la misma persona. En caso contrario, el sistema determina que x_1 y x_2 son personas diferentes. La idea es pasar dos entradas diferentes sobre la misma red neuronal y comparar las salidas. Esto es lo que se conoce como **redes neuronales siamesas** (Taigman *et al.*, 2014).

Vamos a ver ahora cómo se entrenan este tipo de redes neuronales. Formalmente, queremos aprender los parámetros de tal forma que si dos imágenes de entrada x_i y x_j pertenecen a la misma persona la distancia entre su codificación sea pequeña y, por el contrario, si x_i y x_j son personas diferentes, entonces queremos que la distancia sea grande.

Si variamos los parámetros en las capas de la red neuronal terminamos con diferentes codificaciones. Podemos usar el algortimo de entrenamiento *backpropagation* y modificar todos los parámetros para asegurar que se satisfacen estas condiciones. Para que la red aprenda lo que acabamos de describir, debemos definir una función objetivo, que en este caso será la función *triplet loss*.

4.1.1. Triplet loss

Una buena forma de aprender los parámetros de la red neuronal, para que nos dé una buena codificación para las imágenes de las caras, es aplicar el algoritmo del descenso del gradiente sobre la función *triplet loss*.

Para aplicar el *triplet loss* necesitamos comparar pares de imágenes. En la terminología de *triplet loss* identificamos a una imagen como *anchor* y buscamos que la distancia entre el *anchor* y la imagen positiva sea pequeña, mientras que queremos que la distancia entre la imagen *anchor* y la imagen negativa sea más grande. Esto es lo que da nombre al termino *triplet loss*, que siempre estaremos mirando a tres imágenes al mismo tiempo: *anchor* (A), positiva (P) y negativa (N). Para formalizar este principio que acabamos de comentar, queremos que los parámetros de la red neuronal tengan la siguiente propiedad para las codificaciones que produce:

$$\|f(P) - f(A)\|^2 \leq \|f(P) - f(N)\|^2. \quad (4)$$

Lectura complementaria

Y. Taigman, M. Yang, M. Ranzato y L. Wolf. «DeepFace: Closing the Gap to Human-Level Performance in Face Verification». 2014 IEEE Conference on Computer Vision and Pattern Recognition, 2014, p. 1701-1708, doi: 10.1109/CVPR.2014.220.

Para asegurar que se cumple la ecuación anterior y que la salida de las codificaciones no es siempre 0, vamos a aprender y optimizar la siguiente ecuación:

$$\|f(P) - f(A)\|^2 - \|f(P) - f(N)\|^2 + \alpha \leq 0, \quad (5)$$

donde α es un hiperparámetro (*margin*) que previene que las salidas no sean optimizadas de forma eficiente, intentando separar los pares $A - P$ de $A - N$.

Para formalizar todo esto vamos a definir el *loss* de la siguiente forma:

$$\mathcal{L}(A, N, P) = \max(\|f(P) - f(A)\|^2 - \|f(P) - f(N)\|^2 + \alpha, 0) \quad (6)$$

Y la función de coste global de la red neuronal se puede definir como:

$$J = \sum_{i=1}^m \mathcal{L}(A_i, P_i, N_i). \quad (7)$$

Si, por ejemplo, tenemos un conjunto de datos de entrenamiento de 10.000 imágenes con 1.000 personas diferentes, lo que haremos es usar las 10.000 imágenes para generar *triplets*. A continuación, entrenaremos la red utilizando estos *triplets* y usando el método del descenso del gradiente sobre la función de coste que hemos definido anteriormente.

El conjunto de datos de entrenamiento debe contener varias imágenes de la misma persona, ya que debemos generar los pares $A - P$. Sin embargo, después de la fase de entrenamiento, la red solo necesita una imagen de cada persona.

En este punto, nos surge la duda de cómo seleccionar las imágenes que formarán los *triplets*. Si seleccionamos A , P y N de forma aleatoria (siendo P una imagen que contiene la misma persona que A , y N una persona distinta), la restricción es muy fácil de satisfacer. Lo que queremos es seleccionar el *triplet* para que sea difícil de entrenar. En concreto, queremos que para todos los *triplets*, la distancia $d(A, P)$ sea muy próxima a $d(A, N)$. Si no los escogemos de esta forma, habrá muchos *triplets* que pueden satisfacer la función de coste y, por lo tanto, el método del descenso del gradiente no hará nada debido a que la red neuronal generará la salida deseada la mayoría de las veces (Schroff *et al.*, 2015).

Lectura complementaria

F. Schroff, D. Kalenichenko y J. Philbin. «FaceNet: A unified embedding for face recognition and clustering». 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, p. 815-823, doi: 10.1109/CVPR.2015.7298682.

4.2. Transferencia de estilos

Imaginemos que tenemos una imagen de un gato y queremos obtener esta misma imagen, pero con un estilo diferente. Por ejemplo, queremos obtener la misma imagen del gato, pero con un estilo como si hubiese sido pintado por Van Gogh. Lo que nos permite la transferencia de estilos es, precisamente, esto: **generar una imagen nueva a partir de una imagen de origen, pero aplicando el estilo de una segunda imagen**. La figura 17 muestra el ejemplo que

acabamos de comentar. Para lo que sigue de sección vamos a denominar C la imagen contenido, S la imagen de estilo y G la imagen que se genera.

Figura 17. Ejemplo de transferencia de estilo



Para poder hacer este proceso, necesitamos mirar las características extraídas por la CNN en varias capas, de menos a más profundas.

En primer lugar, vamos a definir una función de coste $J(G)$ que nos dirá cómo de buena es una imagen generada en particular. Vamos a usar el método del descenso del gradiente para minimizar la función $J(G)$. Vamos a dividir esta función en dos partes:

- **Coste de contenido:** $J_{content}(C, G)$. Esta función mide el grado de similitud entre el contenido de la imagen generada (G) y la imagen contenido (C).
- **Coste de estilo:** $J_{style}(S, G)$. Calcula el grado de similitud en el estilo entre la imagen de estilo (S) y la imagen generada (G).

Estas dos funciones de coste se suman y ponderan con dos hiperparámetros, α y β , que especifican el peso relativo de ambas funciones de coste para determinar el coste total (Gatys *et al.*, 2016):

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G) \quad (8)$$

El algoritmo funciona de la siguiente forma: inicializamos de forma aleatoria la imagen G y usamos el método del descenso del gradiente para minimizar la función $J(G)$, de manera que se actualizan los valores de los píxeles. Vamos a ver con más detalle cómo se definen las dos funciones de coste.

4.2.1. Coste de contenido

Supongamos que vamos a utilizar una capa oculta (l) para calcular el coste de contenido. Entonces, si l es una capa inicial de la red, es decir, próxima

Lectura complementaria

L. A. Gatys, A. S. Ecker y M. Bethge. «Image Style Transfer Using Convolutional Neural Networks». 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, p. 2414-2423, doi: 10.1109/CVPR.2016.265.

a los datos de entrada, entonces estamos forzando que la imagen generada tenga valores de píxeles muy similares a los de la imagen de contenido. Mientras que si usamos una capa más profunda, es decir, próxima a la salida de la red, estaremos forzando que si hay un gato en la imagen contenido, deberá aparecer el gato también en la imagen generada.

Una forma de conseguir este valor de coste es usar una red neuronal preentrenada, como por ejemplo VGG, y medir el grado de similitud de contenido entre la imagen de contenido y la imagen generada. Supongamos que $a_C^{[l]}$ y $a_G^{[l]}$ son las activaciones para la capa l . Si estas dos activaciones son similares, implica que las dos imágenes tienen un contenido similar. Vamos pues a definir la función de coste $J_{content}(C, G)$ para que nos calcule el grado de disimilitud entre los valores de las activaciones:

$$J_{content}(C, G) = \frac{1}{2} \|a_C^{[l]} - a_G^{[l]}\|^2 \quad (9)$$

4.2.2. Coste de estilo

Imaginemos que hemos escogido una capa oculta (l) para medir el estilo de la imagen. Lo que debemos hacer es definir el estilo como la correlación entre las activaciones a través de los diferentes canales de esta capa de activación l . La intuición nos sugiere usar el grado de correlación entre canales como medida del estilo. Si calculamos, por ejemplo, el grado de correlación entre el primer y el segundo canal de la imagen generada, esto nos dará información sobre la frecuencia de aparición de una textura concreta y, por lo tanto, podremos medir el grado de similitud en el estilo entre la imagen generada y la imagen de estilo.

Para formalizar esta intuición vamos a generar una matriz, a la que llamaremos **matriz de estilo**, que medirá todas las correlaciones que hemos comentado anteriormente.

Nos referiremos a $a_{i,j,k}^{[l]}$ como el valor de activación en la posición i, j, k de la capa de activación l , de tal forma que i indexa en altura, j indexa en el ancho y k indexa a través de los diferentes canales.

A continuación, vamos a calcular una matriz $G^{[l]}$, que será de dimensión $n_c \times n_c$, donde n_c es el número de canales existentes en la capa l , y que medirá el grado de correlación entre cada par de canales. En particular, $G_{kk'}^{[l]}$ medirá cómo de correlacionados están los valores de activación del canal k con el canal k' , donde k y $k' \in \{1, \dots, n_c\}$.

Esta matriz la calculamos para la imagen estilo (S) y para la imagen generada (G) de la siguiente forma:

$$G_{kk'}^{[l](S)} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{i,j,k}^{[l](S)} a_{i,j,k'}^{[l](S)} \quad (10)$$

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{i,j,k}^{[l](G)} a_{i,j,k'}^{[l](G)} \quad (11)$$

Si los valores de activación están correlacionados, entonces el valor de $G_{kk'}$ será alto; mientras que si no lo están, será un valor pequeño.

Finalmente, la función de coste sobre una capa l se define de la siguiente forma:

$$J_{style}^{[l]}(S, G) = \frac{1}{(2n_H^l n_W^l n_C^l)^2} \sum_k \sum_{k'} (G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)}) \quad (12)$$

Y la función de coste de estilo global se define como:

$$J_{style}(S, G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S, G) \quad (13)$$

donde $\lambda^{[l]}$ es un hiperparámetro.

5. Consejos prácticos

En este capítulo nos centraremos en ver algunos consejos prácticos a la hora de trabajar con CNN e imágenes para la creación de modelos predictivos.

En esta sección resumiremos algunos consejos prácticos, frutos de varios años trabajando con redes neuronales convolucionales en temas de procesamiento de imágenes y visión por ordenador, que pueden resultar muy útiles a la hora de abordar diferentes proyectos relacionados con la creación de modelos predictivos empleando las CNN.

Hemos visto hasta el momento diferentes arquitecturas muy efectivas de redes convolucionales. En esta sección vamos a ver algunos consejos prácticos de cómo usarlas, empezando primero con las implementaciones de código abierto (*open source*).

Muchas de las redes neuronales convolucionales conocidas y empleadas actualmente son muy difíciles de replicar, debido principalmente a dos motivos: (1) los detalles de implementación no son conocidos o no son fáciles de replicar, y (2) el ajuste (*tuning*) de los hiperparámetros, como por ejemplo el *learning decay* que pueden marcar una diferencia importante en el rendimiento del modelo. Replicar una CNN a partir de un artículo científico (o similar) es una tarea compleja y difícil, incluso para muchos expertos, simplemente a partir de la información descrita en el artículo del trabajo de referencia.

Por suerte, muchos investigadores en el área de *deep learning* abren su código a internet. Es decir, publican en código de forma abierta en ciertos repositorios, como por ejemplo GitHub⁽¹⁾ u otros repositorios de código similares.

Entonces, si vemos un artículo de investigación, los resultados del cual queremos replicar, una primera opción que debemos considerar es buscar una implementación *open source* disponible en la red del modelo en cuestión. Si podemos obtener la implementación de los autores avanzaremos mucho más rápido que si queremos implementar el mismo método desde cero. Aunque también hay que tener en cuenta que implementar desde cero un modelo es un muy buen ejercicio para entenderlo.

Veamos un ejemplo. Imaginemos que queremos usar las *residual networks* (ResNets) y hacemos una búsqueda en internet con los conceptos «ResNet github», a partir de la cual nos aparecen muchos enlaces a diferentes implementaciones.

GitHub

Es una plataforma de desarrollo colaborativo para alojar proyectos utilizando el sistema de control de versiones Git ampliamente empleado por la comunidad científica, entre muchas otras.

⁽¹⁾<https://github.com/>

Uno de los enlaces que probablemente nos aparecerá (y probablemente en primera posición) es la implementación original de los autores del artículo ResNet⁽²⁾. Si accedemos al código de GitHub veremos la descripción del trabajo de la implementación en particular. Veremos también la licencia del código, en este caso concreto, corresponde a una licencia MIT para poder ver las implicaciones de usar el código. La licencia MIT es una de las más permisivas y abiertas. A partir del repositorio, podemos clonar el repositorio de forma local, es decir, en el sistema de ficheros de nuestro ordenador. En este caso particular, la implementación usa el *framework* de computación Caffe, pero si se precisa una implementación de este modelo en otro *framework* (e.g. Keras) es muy probable que también esté disponible de forma abierta en algún repositorio público de código.

⁽²⁾<https://github.com/KaimingHe/deep-residual-networks>

Si estamos desarrollando una aplicación de visión por ordenador, un flujo de trabajo (*workflow*) muy común podría ser:

- 1) seleccionar la arquitectura que nos gustaría probar,
- 2) buscar una implementación de código abierto en cualquier repositorio de código *open source*
- 3) y empezar a construir el modelo a partir de la implementación seleccionada.

Una de las ventajas de trabajar de esta forma es que las redes neuronales profundas tardan mucho tiempo en entrenar y quizás alguien haya entrenado el mismo modelo usando GPU con un conjunto de datos lo suficientemente grande.

Esto nos permitiría usar el modelo ya entrenado o bien usar técnicas de *transfer learning* (véase sección 2). Por supuesto si estamos haciendo investigación en redes neuronales lo que queremos es hacer una implementación desde cero y el flujo de trabajo sería muy diferente.

Resumen

En este texto hemos presentado las principales arquitecturas y aplicaciones de las redes neuronales convolucionales (en inglés, *Convolutional Neural Networks*, CNN).

Inicialmente, hemos revisado algunas de las principales arquitecturas de CNN, incluyendo las redes clásicas que han sentado las bases para el desarrollo de otras arquitecturas más complejas y modernas. Como ya hemos comentado, una revisión exhaustiva de arquitecturas de redes convolucionales queda totalmente fuera del alcance de este texto.

En segundo lugar, hemos introducido el concepto de la «transferencia del aprendizaje» (en inglés, *transfer learning*) que nos permite emplear modelos preentrenados y beneficiarnos de modelos entrenados con grandes conjuntos de datos.

En el tercer capítulo hemos tratado el aumento de datos (*data augmentation*), que también es conocido como la generación artificial de datos o construcción de datos sintéticos. Este proceso nos permite generar más datos de entrenamiento a partir de nuestros datos disponibles, con todas las ventajas que esto puede aportar al entrenamiento y validación de un modelo.

El cuarto bloque de este módulo se ha centrado en algunas aplicaciones relevantes en el ámbito de las CNN, como son el reconocimiento facial y la transferencia de estilo.

Finalmente, hemos presentado un último capítulo dedicado a sugerir algunas buenas prácticas y consejos para el desarrollo e implementación de CNN en aplicaciones reales.

Glosario

aprendizaje automático *m* El aprendizaje automático (más conocido por su denominación en inglés, *machine learning*) es el conjunto de técnicas, métodos y algoritmos que permiten a una máquina aprender de manera automática en base a experiencias pasadas.

aprendizaje no supervisado *m* El aprendizaje no supervisado se basa en el descubrimiento de patrones, características y correlaciones en los datos de entrada, sin intervención externa.

aprendizaje supervisado *m* El aprendizaje supervisado se basa en el uso de un componente externo, llamado supervisor, que compara los datos obtenidos por el modelo con los datos esperados por este, y proporciona retroalimentación al modelo para que vaya ajustándose y mejorando las predicciones.

gitHub *f* GitHub (<https://github.com/>) es una plataforma de desarrollo colaborativo para alojar proyectos utilizando el sistema de control de versiones Git ampliamente empleado por la comunidad científica, entre muchas otras.

Bibliografía

Bengio, Y., Goodfellow, I., Courville, A. (2016). *Deep Learning*. Cambridge, MA: MIT Press.

Casas Roma, J., Lozano Bagén, T., Bosch Rué, A. (2020). *'Deep learning': Principios y fundamentos*. Barcelona: Editorial UOC.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2ª edición, O'Reilly Media, Inc.

Haykin, S. O. (2009). *Neural Networks and Learning Machines*. 3ª edición. Pearson.

Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press.